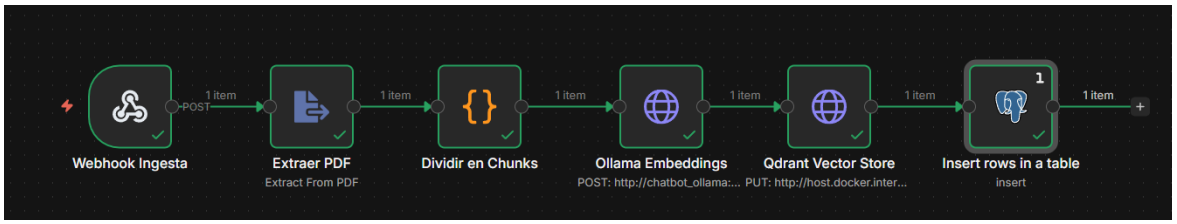
INGESTA WORKFLOW



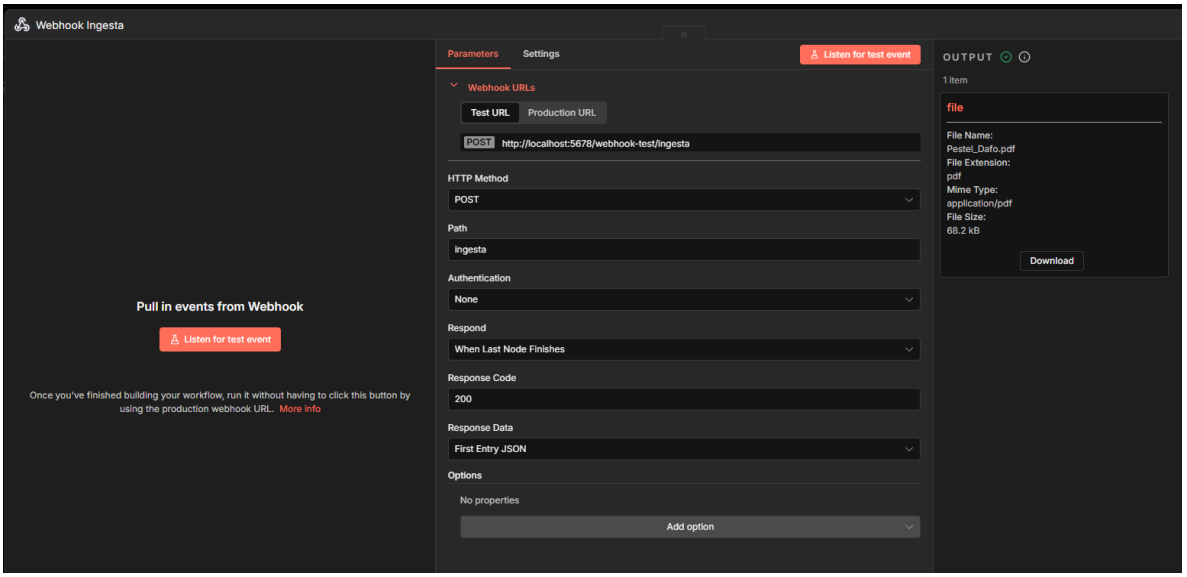
1. Inicialización y Disparo del Flujo (Webhook)

El workflow se activa mediante un nodo **Webhook Ingesta** configurado para recibir peticiones externas. Para poner en marcha el proceso, es necesario realizar una llamada al sistema indicando la **ruta local del fichero** que queremos ejecutar o "ingestar" en nuestra base de datos

En este caso práctico, utilizamos la consola para enviar el documento PDF específico mediante el siguiente comando, donde definimos la ruta exacta de origen:

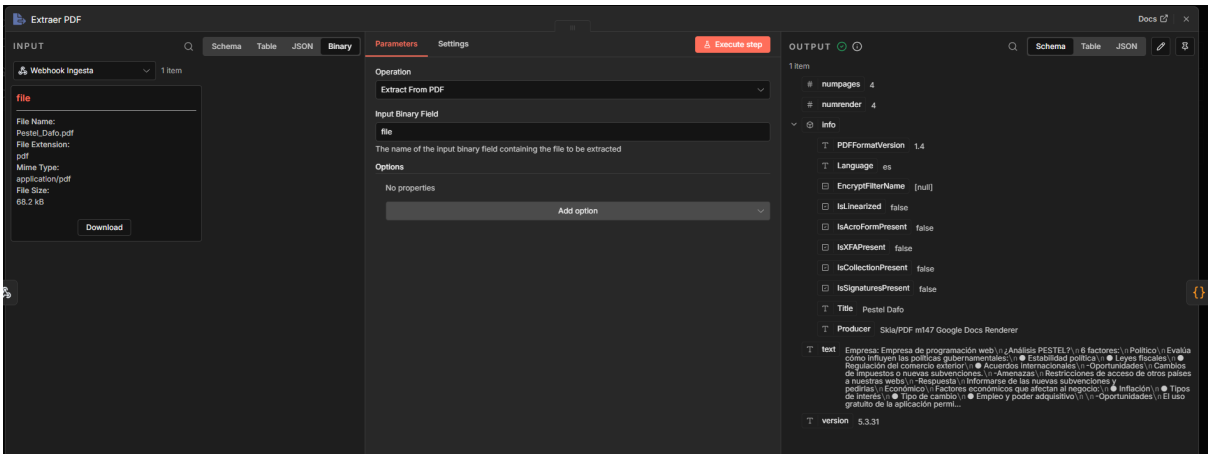
curl -X POST http://localhost:5678/webhook-test/ingesta -F "file=@C:\Users\iruuu\Downloads\Pestel_Dafo.pdf;type=application/pdf".

Al ejecutar este comando, el webhook recibe el archivo binario y comienza automáticamente el proceso de transformación y almacenamiento.



2. Extracción de Contenido (Extraer PDF)
Una vez que el archivo entra por el Webhook, el nodo **Extract From PDF** toma el control

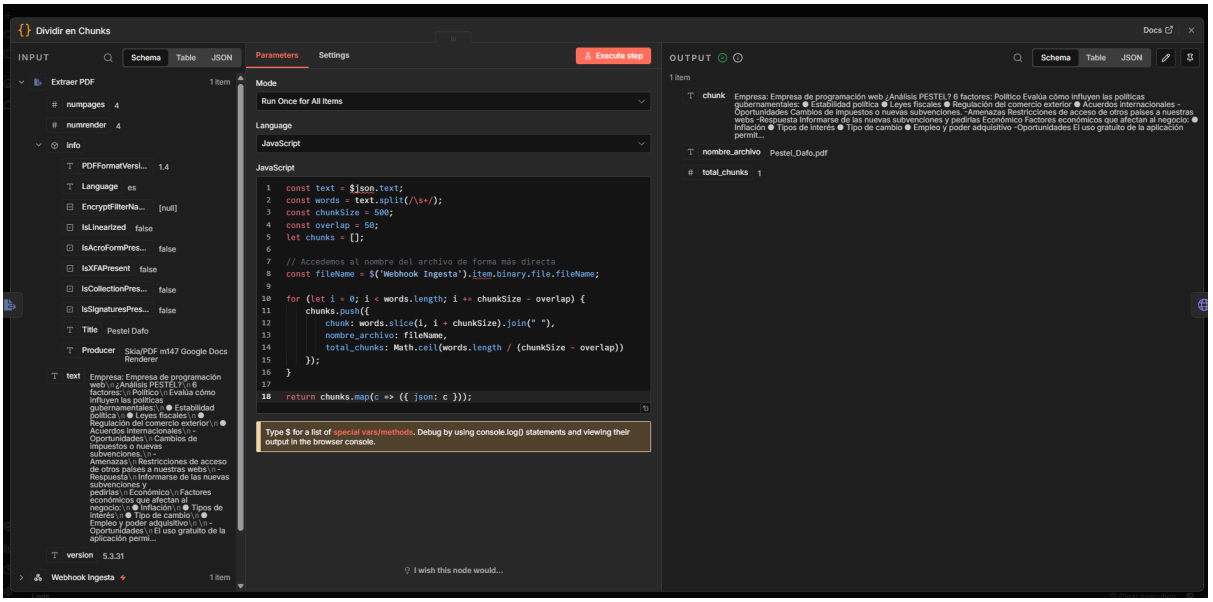
- **Función:** Transforma el archivo binario en texto plano y extrae metadatos clave .
- **Evidencia:** En las capturas se observa que el nodo identifica correctamente el título del documento ("Pestel Dafo") y el texto inicial sobre la "Empresa de programación web".



3. Preprocesamiento de Texto (Dividir en Chunks)

Para optimizar el procesamiento por parte de la IA, el texto se fragmenta mediante un nodo de **Código (JavaScript)**

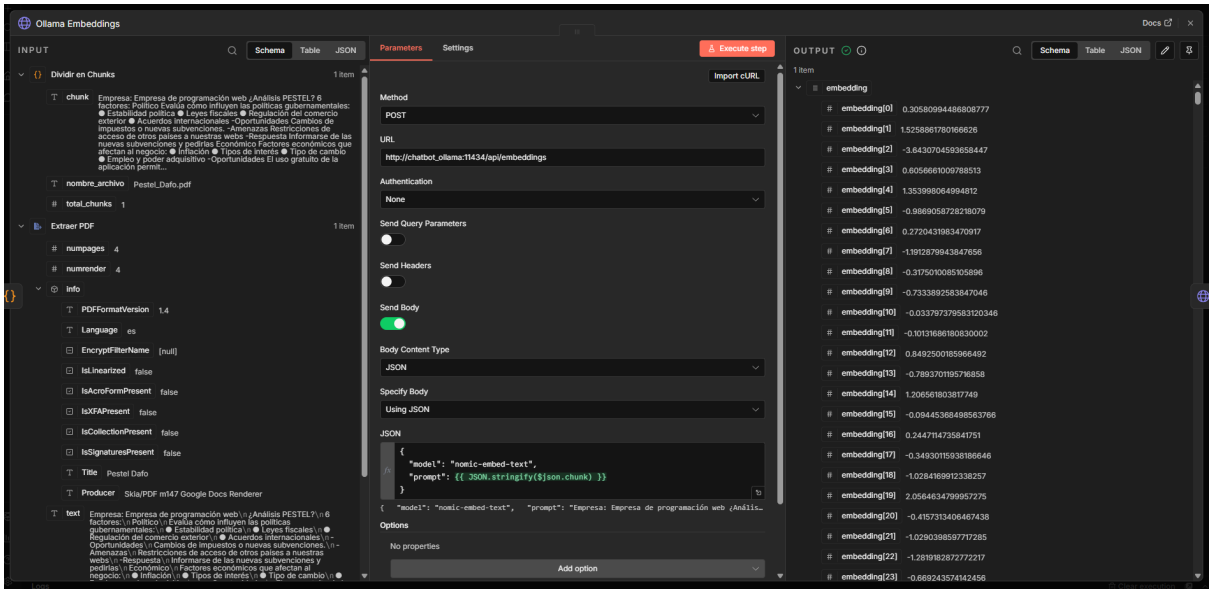
- **Lógica:** Se divide el contenido en bloques (**chunks**) de 500 palabras con un solapamiento de 50 palabras para no perder el contexto entre fragmentos
- **Resultado:** Se genera una estructura que incluye el contenido del bloque, el nombre del archivo original (Pestel_Dafo.pdf) y el cálculo total de fragmentos



4. Generación de Embeddings (Ollama)

Los fragmentos pasan al nodo **Ollama Embeddings**, que actúa como el motor de inteligencia

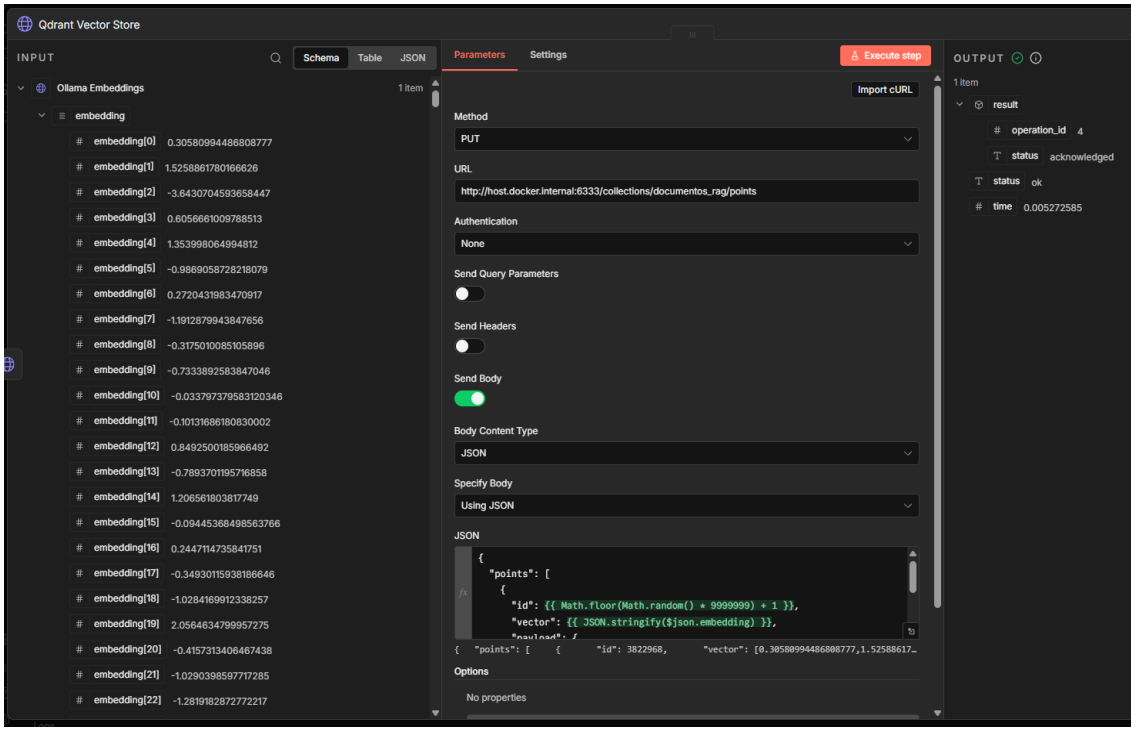
- **Modelo:** Se utiliza nomic-embed-text a través de una API local .
- **Proceso:** Cada fragmento de texto se convierte en un vector numérico (embedding) que permite realizar búsquedas semánticas basadas en el significado y no solo en palabras clave



5. Almacenamiento Vectorial (Qdrant)

Los vectores resultantes se almacenan en la base de datos **Qdrant** .

- **Configuración:** Se realiza una petición PUT a la colección documentos_rag .
- **Estructura:** Se asigna un ID aleatorio a cada punto vectorial para indexar la información de manera que el sistema pueda "recuperarla" eficientemente durante una consulta .

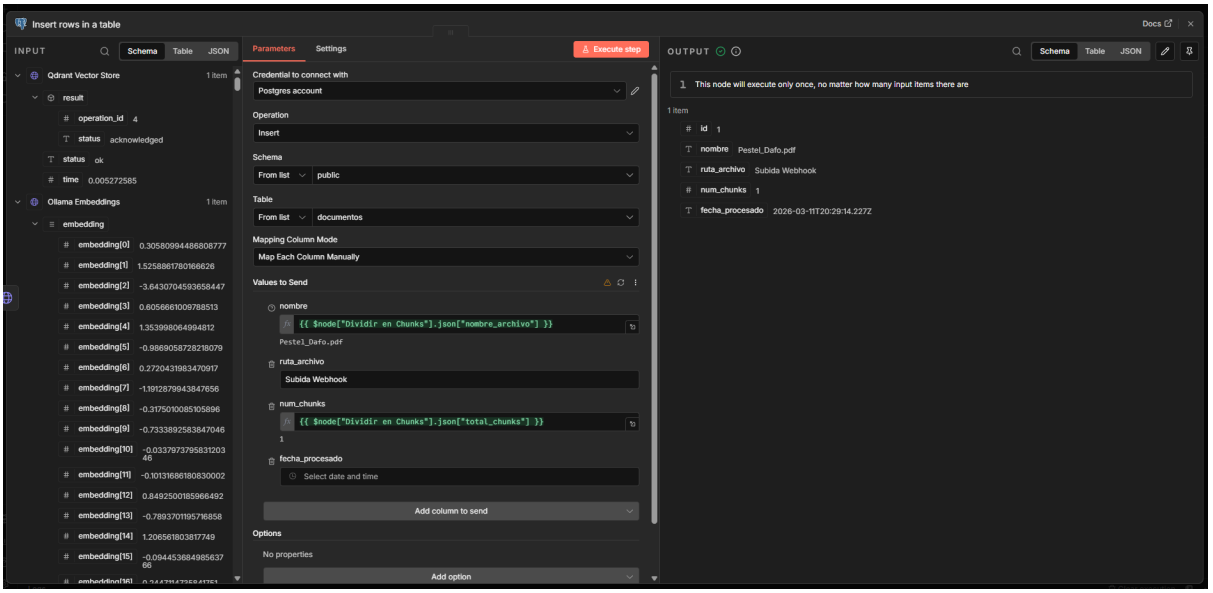


6. Persistencia de Metadatos (PostgreSQL)

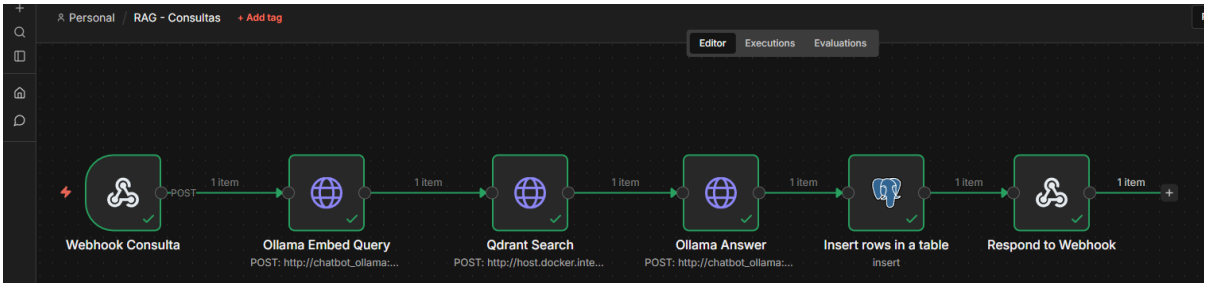
El paso final es el registro administrativo en **PostgreSQL**

- **Operación:** Se inserta una fila en la tabla documentos para llevar un control de trazabilidad
- **Datos registrados:** Se guarda el **nombre del archivo**, la procedencia (Subida Webhook), el **número total de chunks** y la fecha exacta del procesado

Este flujo completo permite transformar un documento estático en una base de conocimientos dinámica lista para ser utilizada en un sistema RAG



CONSULTA WORKFLOW



Documentación del Proyecto: Sistema RAG Educativo (Workflow de Consultas)

1. Descripción General de la Arquitectura

El workflow implementado corresponde al **Workflow 2: Consultas RAG** solicitado en el enunciado. Su objetivo es recibir una pregunta del usuario, recuperar información relevante de una base de datos vectorial y generar una respuesta contextualizada utilizando un modelo de lenguaje local.

La solución utiliza el siguiente stack tecnológico:

- **Orquestador:** n8n.
- **IA (Embeddings y LLM):** Ollama (modelos `nomic-embed-text` y `llama3`).
- **Base de Datos Vectorial:** Qdrant.
- **Persistencia de Datos:** PostgreSQL.

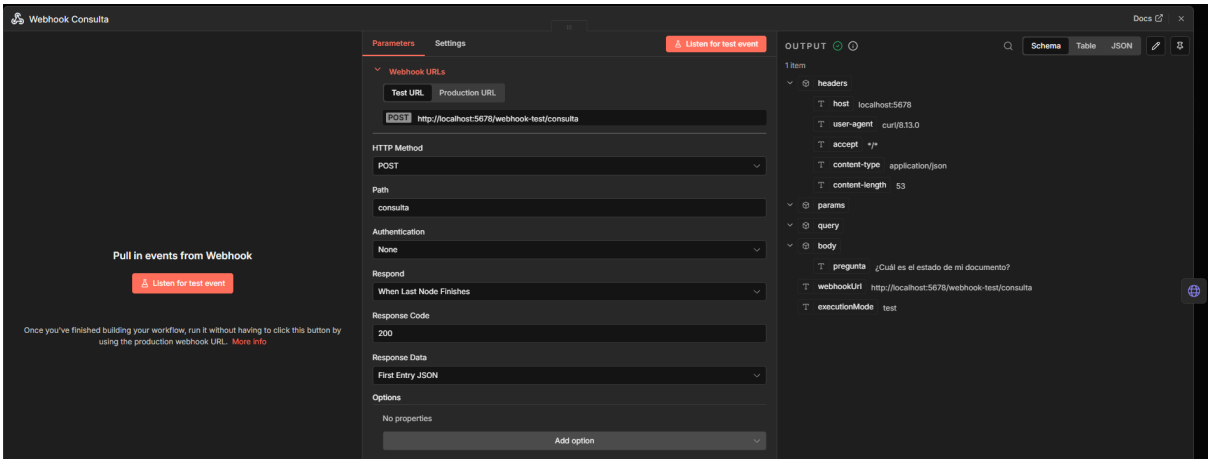
---

2. Análisis Paso a Paso del Workflow

El flujo se compone de **6 nodos principales** interconectados que procesan la información de forma secuencial:

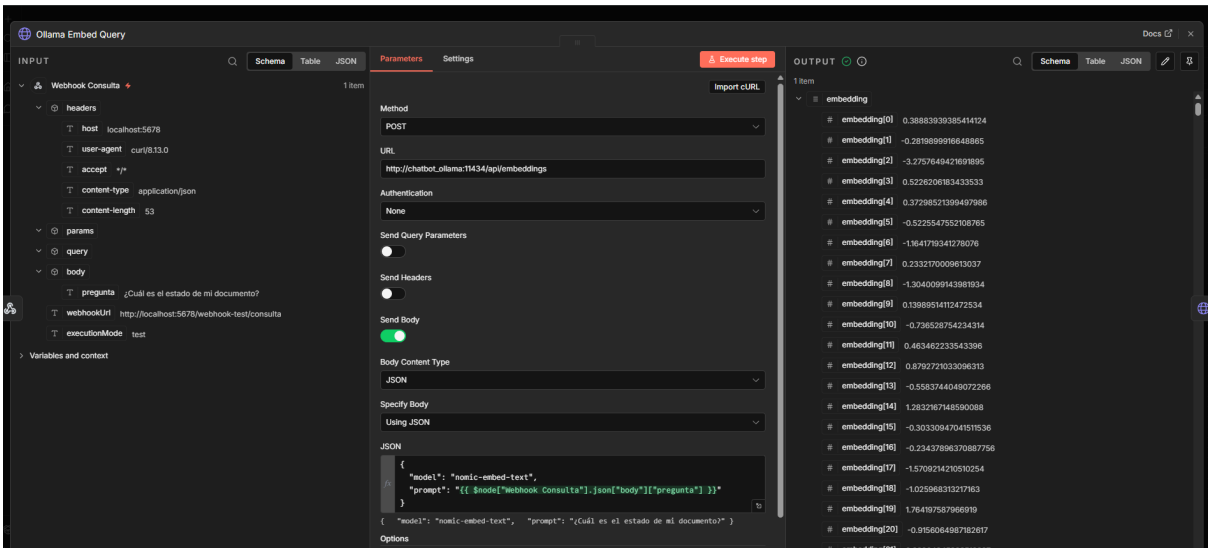
A. Nodo 1: Webhook Consulta (Trigger)

- **Función:** Actúa como el punto de entrada del sistema.
- **Configuración:** Está configurado para escuchar peticiones **HTTP POST** en la ruta `consulta`.
- **Entrada:** Recibe un cuerpo JSON con la pregunta del usuario (ejemplo: *"¿Cuál es el estado de mi documento?"*).



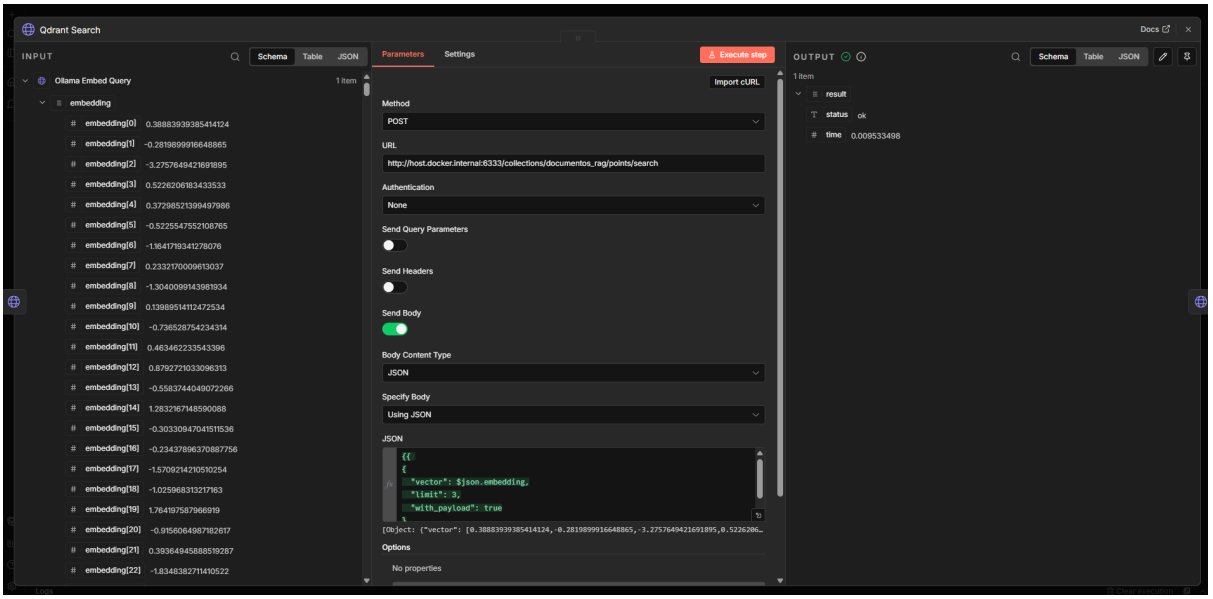
B. Nodo 2: Ollama Embed Query

- **Función:** Convierte la pregunta de texto plano en un vector numérico (embedding) para poder realizar una búsqueda semántica.
- **Configuración:** Realiza una petición POST a la API local de Ollama utilizando el modelo **`nomic-embed-text`**.
- **Proceso:** Toma el valor `{{ $node["Webhook Consulta"].json["body"]["pregunta"] }}` y genera una lista de valores numéricos que representan el significado de la consulta.



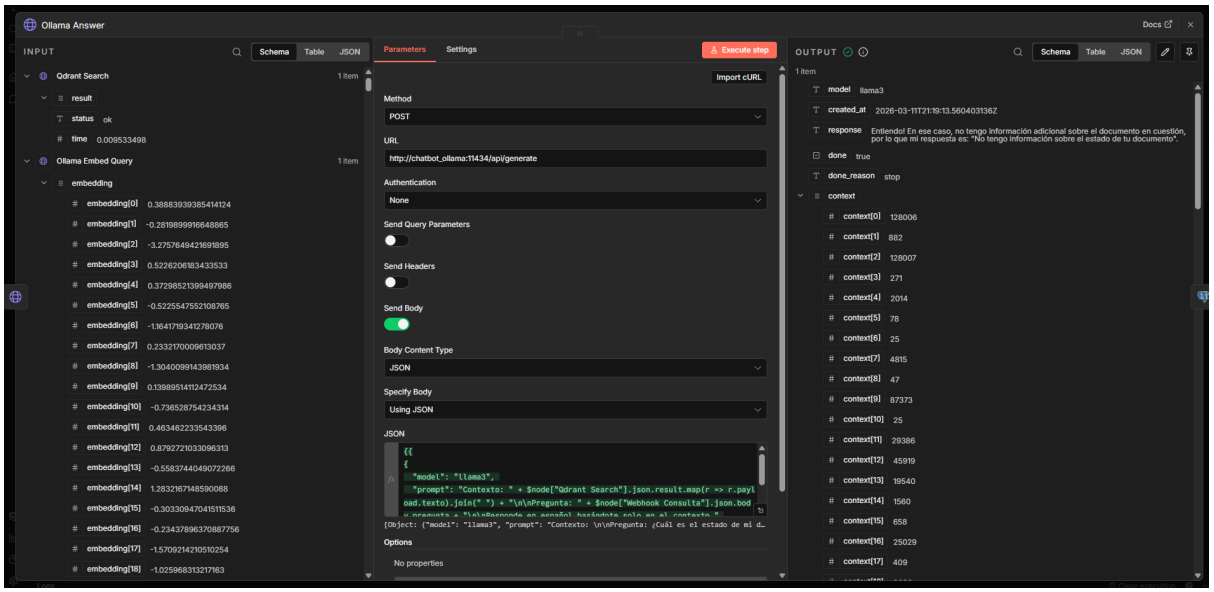
C. Nodo 3: Qdrant Search

- **Función:** Busca en la base de datos vectorial los fragmentos de documentos (chunks) más similares a la pregunta.
- **Configuración:** Envía el vector generado anteriormente a la colección `documentos_rag`.
- **Parámetros:** Está configurado para devolver los **3 resultados más relevantes** (`limit: 3`) e incluir el contenido original (`with_payload: true`).



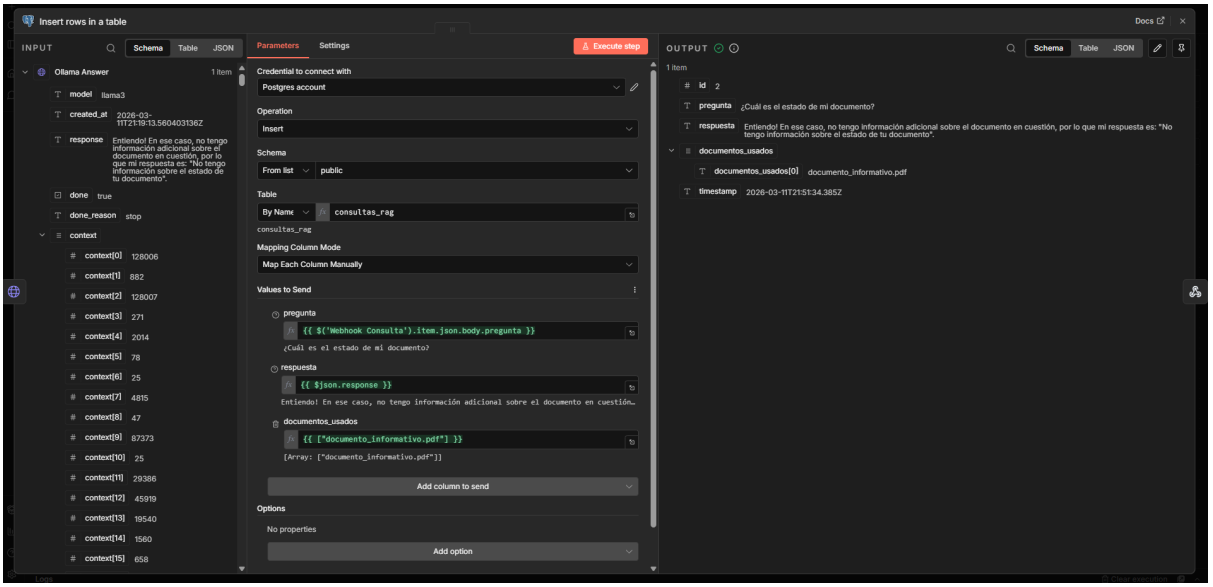
D. Nodo 4: Ollama Answer (Generación de Respuesta)

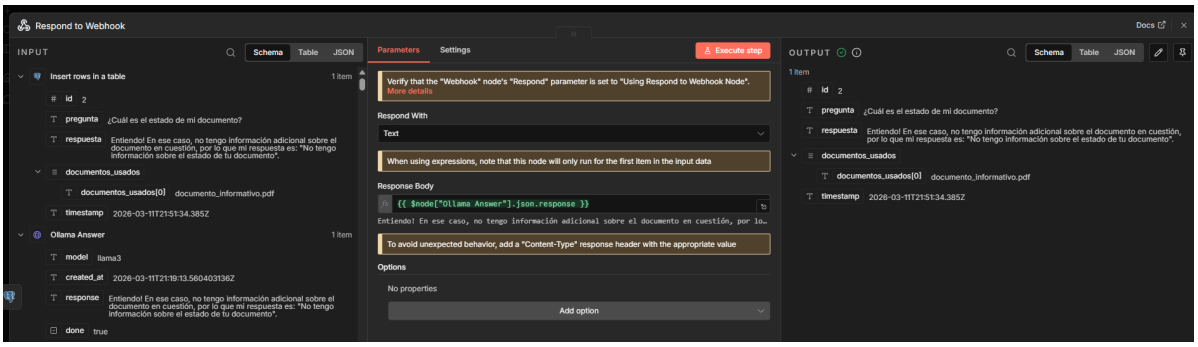
- **Función:** Es el cerebro del sistema. Genera una respuesta final basada en el contexto recuperado.
- **Configuración:** Utiliza el modelo `llama3`.
- **Prompt Engineering:** Se construye un prompt dinámico que combina:
    1. El **Contexto**: Los fragmentos recuperados de Qdrant.
    2. La **Pregunta**: La consulta original del usuario. *Instrucción:* El modelo debe responder exclusivamente basándose en el contexto proporcionado.



E. Nodo 5: Insert rows in a table (PostgreSQL)

- **Función:** Cumple con el requisito obligatorio de mantener un **historial de consultas**.
- **Configuración:** Conecta con una base de datos PostgreSQL y utiliza la tabla `consultas_rag` definida en el esquema del hito.
- **Datos persistidos:** Se guardan la `pregunta`, la `respuesta` generada y los `documentos_usados` (en este caso, `documento_informativo.pdf`) junto con una marca de tiempo.



F. Nodo 6: Respond to Webhook

- **Función:** Cierra el ciclo de comunicación devolviendo la respuesta final al cliente que realizó la consulta.
- **Salida:** Devuelve el texto generado por el modelo `llama3` contenido en `{{ $node["Ollama Answer"].json.response }}`.